Lec-19

12/07/26 2:13 PM

Sliding-window

① Array | String K → output { Sub Array, Sub string }

number

Condition

Sliding-window → Fixed size, Variable size

[2,1,3,5,7,6,4,3,2,1] K=3

n, k 10^5

T.C=TLE

2 1 3
1 3 5
3 5 7
5 7 6
7 6 4
6 4 3
4 3 2
3 2 1

```
for(i=0  i<n  i++){
for(j=i  j<i+K  j++){
}
}
```

① Fixed → 1st window

ans = ~~6~~ ~~9~~ ~~15~~ 18

[2,1,3,5,7,6,4,3,2,1] K=3
0 1 2

```
(i) for(i=0 i<K i++){
      sum += arr[i]
    }
    ans=sum
(ii) for(i=K i<n i++){
      sum += arr[i]
      sum -= arr[i-K]
      ans = Math.max(ans, sum)
    }
```

sum = 6 + 5 − 2
= 9+7−1
= 15+6 −3
= 18 + 4−5
= 17+3−7
= 13+2−6
= 9+1−4
= 6

(ii)
→ window grow
→ window shrink
→ ans update

1 → (1)
2 [2, 1 2]
3 [4, 2 4, 1 2 4]
1 [3]
2 [2, 3 2]

[1 2 4 3 2] K=10

24/1 = 24
24/2 = 12
12/4 = 3

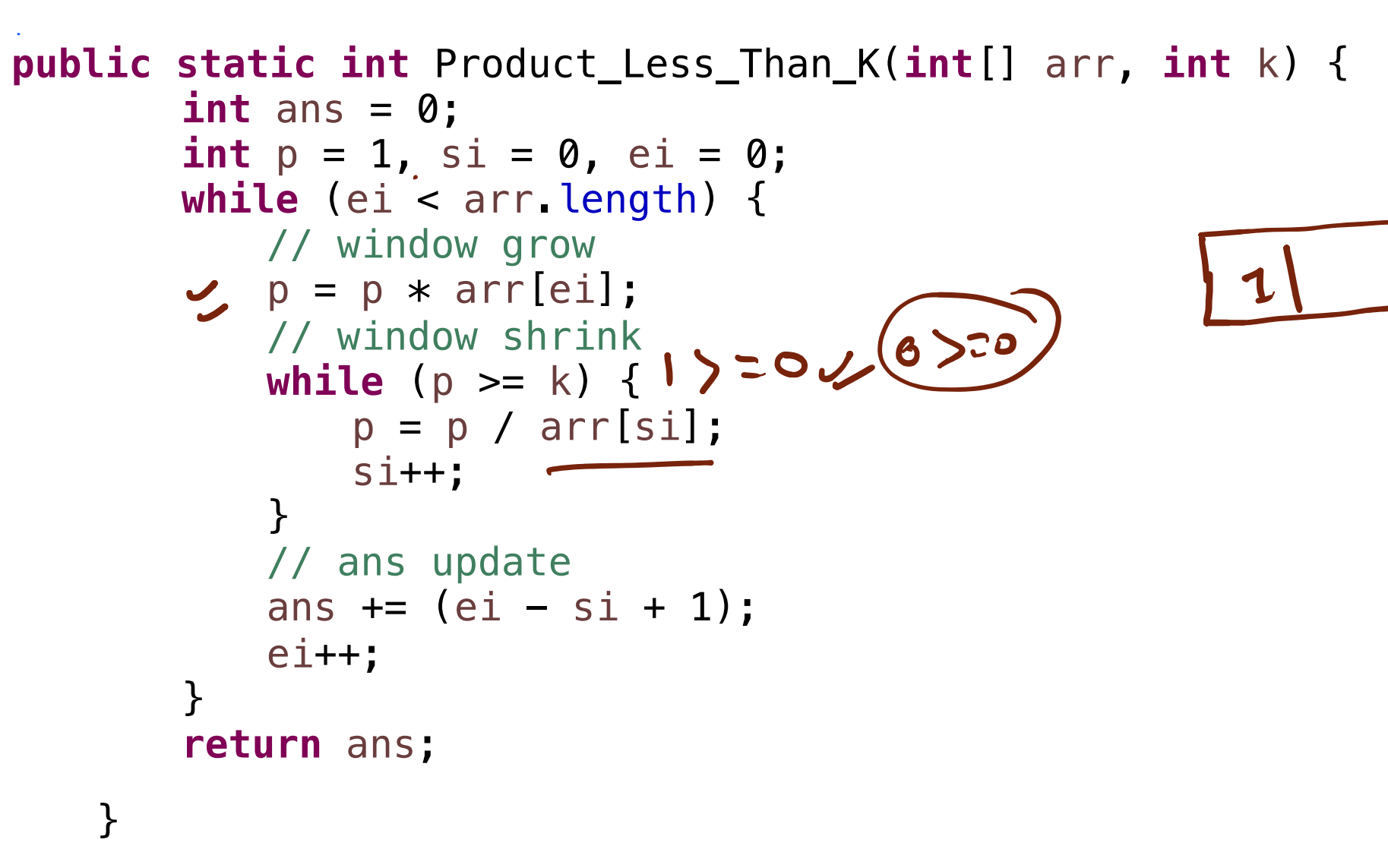
ans = 1+2+3+1+2

si
0 1 2 3 4
[1 2 4 3 2] K=10
ei

ans=0, P=1

P = P×arr[ei]
= 1×2
= 2×4
= 8×3
= 24/1
= 24/2
= 12/4
= 3×2
= 6

```
while(ei<arr.len){
  1. grow
     p = p×arr[ei]
  2. shrink
  while (P>=K){
     p = p/arr[si]
     si++
  }
  3. ans update
     ans += window size
     ei++
}
```

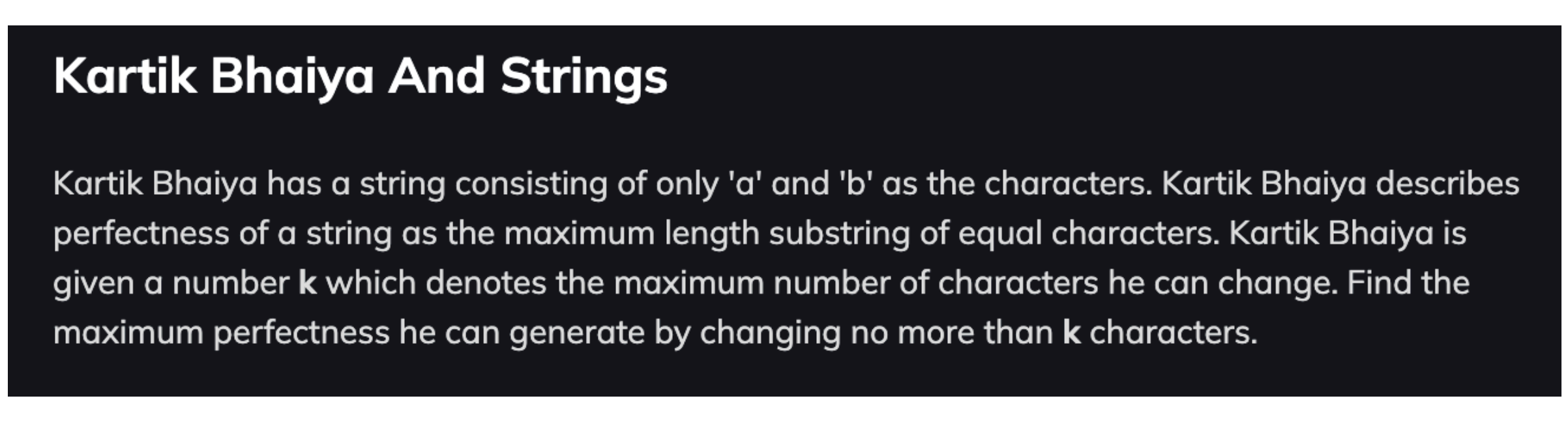


```
public static int Product_Less_Than_K(int[] arr, int k) {
        int ans = 0;
        int p = 1, si = 0, ei = 0;
        while (ei < arr.length) {
            // window grow
            p = p * arr[ei];
            // window shrink
            while (p >= k) {
                p = p / arr[si];
                si++;
            }
            // ans update
            ans += (ei - si + 1);
            ei++;
        }
        return ans;
    }
```

1>=0 ✓ 0>=0

[1, 2, 3] K=0
si ei

P=1/1
= 1/2 => 0
= 0/3 = 0



Kartik Bhaiya And Strings

Kartik Bhaiya has a string consisting of only 'a' and 'b' as the characters. Kartik Bhaiya describes perfectness of a string as the maximum length substring of equal characters. Kartik Bhaiya is given a number **k** which denotes the maximum number of characters he can change. Find the maximum perfectness he can generate by changing no more than **k** characters.

aaab aaab aaab aabaa K=2

ch='b'

flip = ~~X~~ ~~X~~ 2

```
si=0, ei=0   flip=0
while(ei<s.len()){
  // grow
  if(s.charAt(ei)==ch){
     flip++
  }
  // shrink
  while(flip>K && si<=ei){
     if(s.charAt(si)==ch){
        flip--
     }
     si++
  }
  // ans
  ans = max(ans, ei-si+1)
  ei++
}
```

```
int a = -∞
(i) for(i=0 i<K i++){
      a = max(a, arr[i])
    }
    ans=a
(ii) for(i=K i<n i++){
      [ ] em K window max
    }
```

→ window grow
→ window shrink
→ ans update

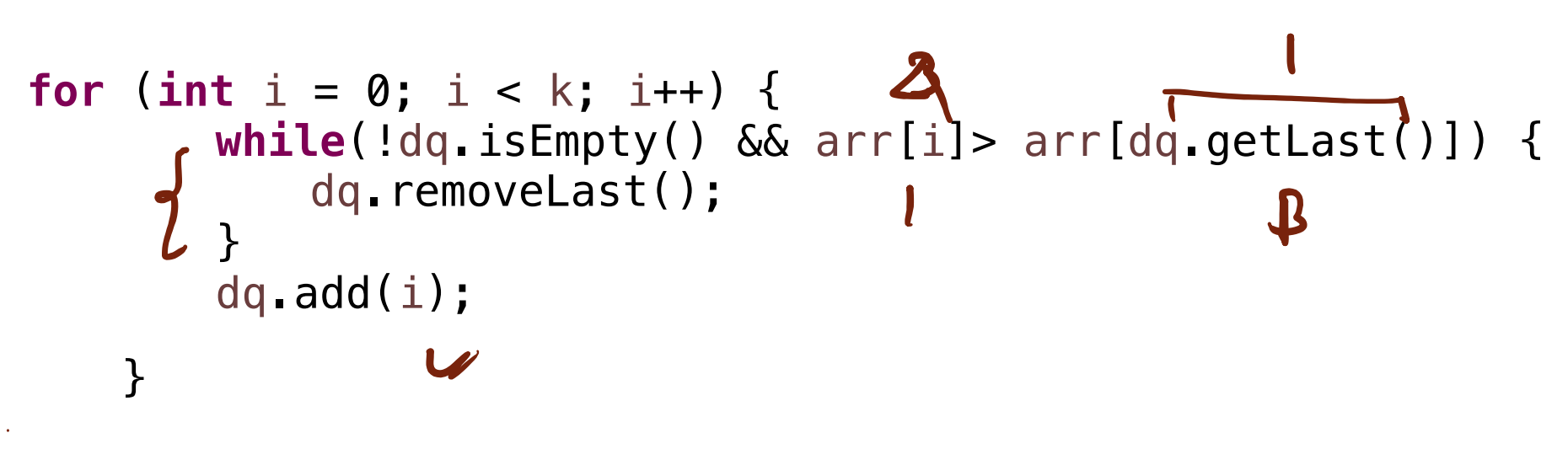
1,3,1,4,−3,−5,−7,−8,3,6,7,2 K=4
0 1 2 3 4 5 6 7 8 9 10 11

0 1 2 3 4 5 6 7 8 9 10 11
[1, 3, 1, 4, -3, -5, -7, -8, 3, 6, 7, 2] K=4

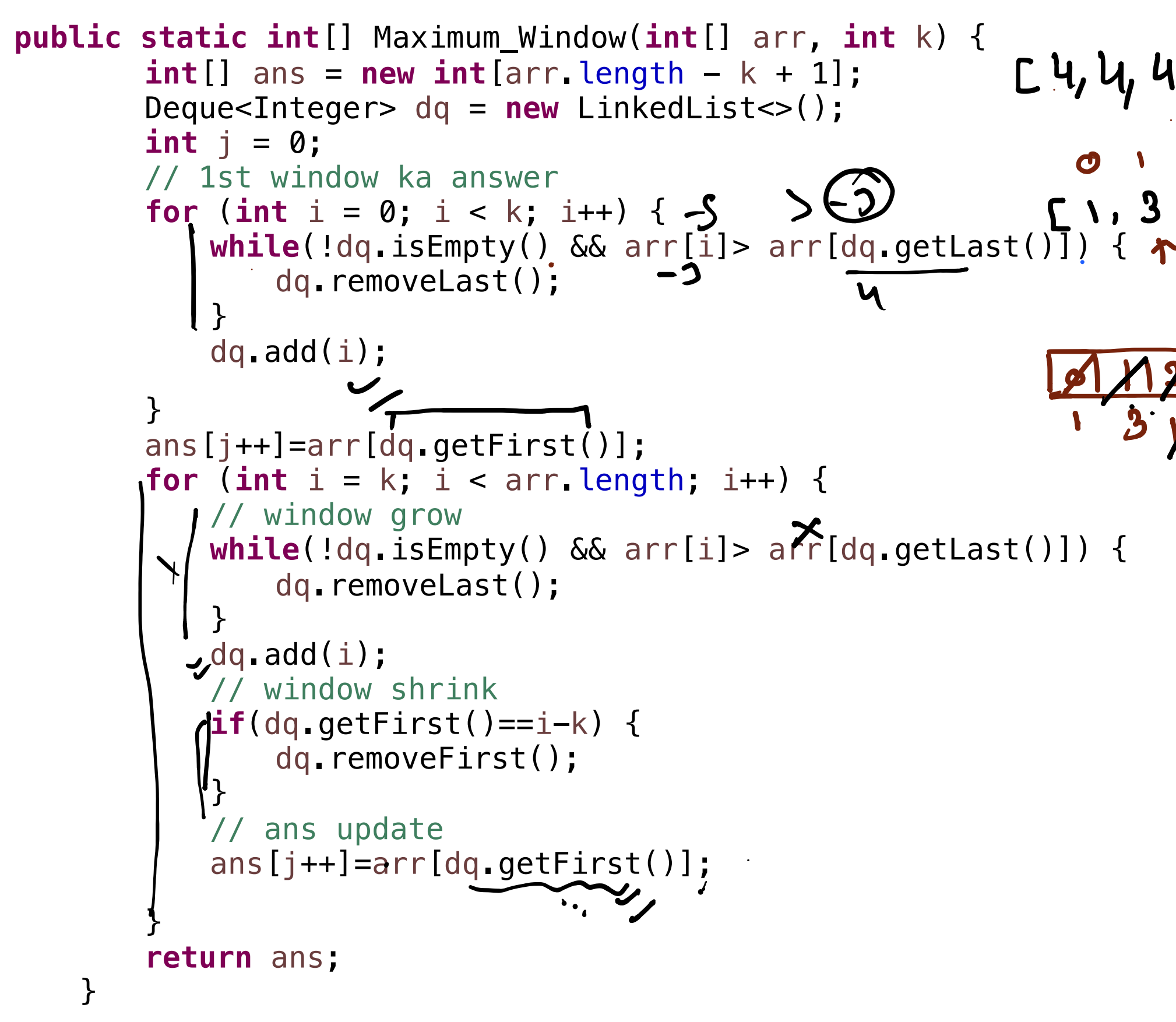
i−K
5−4=1
6−4=2
7−4=3

[1, 3, 1, 4] → 4
[3, 1, 4, -3] → 4
[1, 4, -3, -5] → 4
[4, -3, -5, -7] → 4
(-3, -5, -7, -8) → 3
(-5, -7, -8, 3) → 3
[-7, -8, 3, 6] → 6
(-8, 3, 6, 7) → 7
(3, 6, 7, 2) → 7

0 1 2 3 4 5 6 7 8 9 10 11
[1, 3, 1, 4, -3, -5, -7, -8, 3, 6, 7, 2] K=4



```
for (int i = 0; i < k; i++) {
    while(!dq.isEmpty() && arr[i]> arr[dq.getLast()]) {
        dq.removeLast();
    }
    dq.add(i);
}
```



```
public static int[] Maximum_Window(int[] arr, int k) {
        int[] ans = new int[arr.length - k + 1];
        Deque<Integer> dq = new LinkedList<>();
        int j = 0;
        // 1st window ka answer
        for (int i = 0; i < k; i++) {
            while(!dq.isEmpty() && arr[i]> arr[dq.getLast()]) {
                dq.removeLast();
            }
            dq.add(i);
        }
        ans[j++]=arr[dq.getFirst()];
        for (int i = k; i < arr.length; i++) {
            // window grow
            while(!dq.isEmpty() && arr[i]> arr[dq.getLast()]) {
                dq.removeLast();
            }
            dq.add(i);
            // window shrink
            if(dq.getFirst()==i-k) {
                dq.removeFirst();
            }
            // ans update
            ans[j++]=arr[dq.getFirst()];
        }
        return ans;
    }
```

[4, 4, 4, 4, -3, 3, 6, 7, 7]

K=4

0 1 2 3 4 5 6 7 8 9 10 11
[1, 3, 1, 4, -3, -5, -7, -8, 3, 6, 7, 2] K=4